

Assignatura: Introducció a la Programació

Primers contactes amb la programació: edició, compilació i
execució de codi C en consola.

Facultat de Matemàtiques i Informàtica
Grau d'Enginyeria Informàtica

2 de setembre de 2026



Índex

1	Introducció	3
1.1	Per què programarem des de la consola?	3
1.2	Què necessitarem?	3
2	El procés de construcció d'un programa en C	3
3	La consola	4
3.1	Obrir la consola	5
3.2	El directori actual	6
3.3	Visualitzar el contingut d'un directori	6
3.4	Crear un directori	6
3.5	Canviar de directori	7
3.6	Eliminar fitxers	7
3.7	Visualitzar el contingut d'un fitxer	7
3.8	Exercici consola	7
4	Edició de fitxers amb vim	8
4.1	Crear un fitxer	8
4.2	Els modes de vim	8
4.3	Desar els canvis	9
4.4	Sortir de vim	9
4.5	Desar i sortir	9
4.6	Sortir sense desar	9
4.7	Ordres útils de vim	9
4.8	Exercici 1 vim	10
4.9	Exercici 2 vim	10
5	Instal·lació del compilador de C	11
5.1	Comprovar la instal·lació	12
5.2	Linux	12
5.3	macOS	12
5.4	Windows	12
6	El primer programa en C	12
6.1	Etapa d'EDICIÓ: crear i editar el fitxer font	13
6.2	Etapa de COMPILACIÓ	14
6.3	Etapa d'EXECUCIÓ	15
6.4	Errors	15
6.5	Advertències (warnings)	15
6.6	Exercicis	15
7	Funcions	16
7.1	La nostra primera funció en C	16
7.2	Cridar una funció	17
7.3	Exercicis	17
8	Organització del codi en mòduls	17
8.1	Els fitxers d'un mòdul	18
8.2	La capçalera	18
8.3	La implementació	19
8.4	El programa principal	19
8.5	Compilar els fitxers del mòdul	19

9	Compilació separada	19
9.1	Els fitxers objecte	20
9.2	L'enllaçat	21
10	Automatització de la compilació amb make	21
10.1	Què és un Makefile?	21
10.2	El nostre primer Makefile	21
10.3	Compilar el projecte	22
10.4	Netejar el projecte	22
11	Depuració bàsica	22
11.1	Utilitzar <code>printf()</code>	22
12	Guia d'estil	23
12.1	Comentaris	23
12.2	Noms amb sentit	23
12.3	Format del codi	24
12.4	Notació consistent	24

1 Introducció

L'objectiu d'aquest tutorial és que entengueu el procés complet que segueix qualsevol programa des que s'escriu fins que s'executa. Aquest coneixement és fonamental i us serà útil durant tota la carrera, independentment del llenguatge de programació que utilitzeu més endavant.

Durant les primeres setmanes treballarem amb programes molt petits. Inicialment tots els programes estaran formats per un únic fitxer de codi font. A mesura que augmenti la seva complexitat aprendrem a dividir-los en diversos fitxers per facilitar-ne la lectura, el manteniment i la reutilització del codi.

En aquest tutorial aprendreu a:

- editar programes escrits en C utilitzant un editor de text.
- compilar els programes mitjançant el compilador `gcc`.
- interpretar els missatges d'error i advertència del compilador.
- executar els programes des de la consola.
- organitzar programes senzills en un únic fitxer i, posteriorment, dividir-los en diversos mòduls.
- automatitzar la compilació amb `make`.
- fer depuració bàsica.
- bones pràctiques de programació.

1.1 Per què programarem des de la consola?

Els entorns de desenvolupament moderns (IDE-Integrated Development Environment) automatitzen moltes tasques: creen projectes, compilen el programa, executen l'aplicació i fins i tot suggereixen codi automàticament. Tot i que aquestes eines són molt útils, també amaguen molts dels passos que realitza l'ordinador.

Per aquest motiu, durant aquest curs treballarem únicament amb un editor de text i la consola. Això us permetrà comprendre exactament què passa quan es crea un programa i facilitarà l'aprenentatge de conceptes que reutilitzareu posteriorment en altres assignatures. Una vegada dominats aquests conceptes, podreu utilitzar qualsevol entorn de desenvolupament amb una millor comprensió del seu funcionament intern.

1.2 Què necessitarem?

Per poder seguir aquest tutorial només necessitareu:

- un editor de text (en aquest tutorial utilitzarem `vim`, però al laboratori podeu utilitzar `gedit` o similar).
- una consola (o terminal).
- un compilador de C (`gcc` o `clang`).

2 El procés de construcció d'un programa en C

Després de les etapes d'anàlisi del problema i disseny de la solució, comença la programació. Des de que comencem a programar fins que podem executar el programa, aquest passa per diverses etapes. La Figura 1 mostra les etapes de construcció d'un programa en C: edició, compilació, enllaçat, execució, i, si s'escau, depuració. Els comandes `gcc` que inclou la figura es veurem més endavant en aquest tutorial, per ara només és necessari que entengueu les diferents etapes.

Observeu que aquest procés sempre es realitza en el mateix ordre, però es iteratiu. És a dir, si durant la compilació apareixen errors, primer cal corregir-los i tornar a compilar (veure primera fletxa vermella a la Figura 1). Només quan la compilació i l'enllaçat finalitzen correctament és possible executar el programa. Quan durant l'execució apareixen errors, fem la depuració del codi (localitzar els errors) i hem de tornar a l'etapa d'edició per a corregir-los.

1. CODIFICACIÓ/EDICIÓ

L'edició consisteix a escriure el codi font del programa utilitzant un editor de text. Durant aquesta etapa el programa només és text. L'ordinador encara no el pot executar. Els fitxers de codi font en C tenen l'extensió `.c`. Per exemple: `hello.c`.

2. COMPILACIÓ

Una vegada escrit el programa, aquest s'ha de comprovar. El compilador analitza el codi font, comprova que compleixi les regles del llenguatge C i el tradueix a codi objecte (codi no llegible per a les persones). Aquest fitxer s'anomena fitxer objecte (`.o`). Per exemple, detecta **errors** com oblidar un punt i coma, utilitzar una variable que no ha estat declarada, o escriure incorrectament el nom d'una funció. Si el compilador troba errors, el procés s'atura i cal corregir-los abans de continuar.

A més dels errors, el compilador també pot mostrar **advertències** (*warnings*). Una advertència no impedeix generar el programa executable, però indica que hi ha alguna construcció que podria provocar un comportament incorrecte o que no segueix les bones pràctiques de programació.

3. ENLLAÇAT

Els programes no només utilitzen el codi que escrivim nosaltres. Moltes funcions, com ara `printf()` (que mostra text en la pantalla), formen part de biblioteques ja existents. L'**enllaçador** (*linker*) és el programa encarregat d'unir el nostre codi objecte amb aquestes biblioteques per generar finalment un programa executable. Més endavant també veurem que l'enllaçador permet unir diversos fitxers de codi font escrits per nosaltres.

4. EXECUCIÓ

Quan el programa ha estat compilat i enllaçat correctament, es genera un fitxer executable. Aquest executable ja pot ser carregat pel sistema operatiu i executat. Durant l'execució el programa realitza les operacions que hem programat. Pot passar que durant l'execució es produeixi un error, hem de depurar (següent etapa) el codi per tal d'identificar i fixar l'error.

5. DEPURACIÓ

És habitual que un programa compili correctament però no funcioni com esperem. Per exemple, calcula un resultat incorrecte, entra en un bucle infinit, finalitza inesperadament o mostra informació incorrecta. Aquest tipus de problemes s'anomenen **errors lògics** i es manifesten durant l'execució del programa. El procés de localitzar-los i corregir-los rep el nom de **depuració** (*debugging*). Durant el curs aprendrem diferents tècniques per detectar i corregir aquest tipus d'errors.

Abans d'escriure el primer programa en C, veurem com treballar en la consola del sistema operatiu.

3 La consola

La **consola** (o **terminal**) és una aplicació des de la qual podem executar programes i ordres del sistema operatiu (veure la Figura 2). Durant l'assignatura utilitzarem la consola per:

- crear directoris.
- crear i editar fitxers.
- compilar programes.

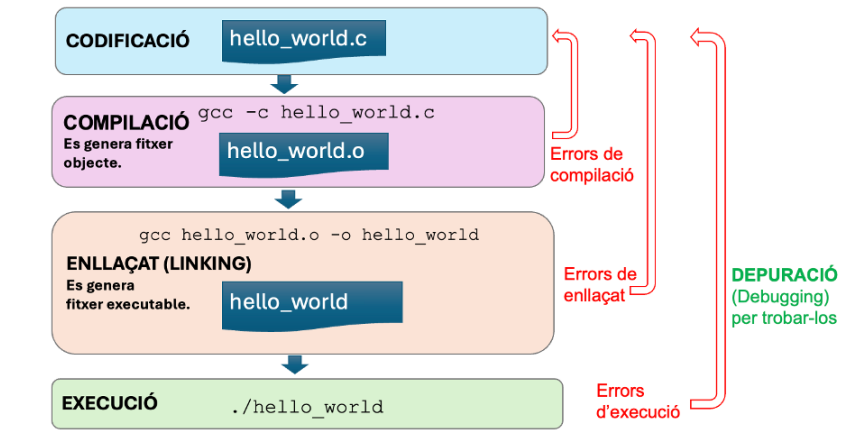


Figura 1: Procés de creació d'un programa en C

- executar programes.

Tot i que més endavant utilitzeu un entorn de desenvolupament (IDE), totes aquestes operacions continuaran existint. L'IDE simplement les executarà de forma automàtica.

3.1 Obrir la consola

Linux

Podeu obrir la consola cercant l'aplicació **Terminal** al menú d'aplicacions.

macOS

Obriu l'aplicació **Terminal**, situada dins el directori (o carpeta) **Applications/Utilities**. Per a usuaris habituals de windows.

Windows

Si instaleu MSYS2 (Minimal SYStem versió 2), que és un entorn mínim que proporciona eines i una capa de compatibilitat tipus Unix/Linux sobre Windows, podeu obrir una consola amb **MSYS2 UCRT64** (Universal C Run Time 64 bits).

Ja oberta la consola, podrem utilitzar una infinitat de comandaments. En aquest tutorial veurem algunes comandaments bàsiques.

Comanda	Descripció
pwd	Mostra el directori actual.
ls	Mostra el contingut del directori.
ls -l	Mostra informació detallada.
mkdir nom	Crea un directori.
cd nom	Entra dins un directori.
cd ..	Torna al directori pare.
cd	Torna al directori personal.
rm fitxer	Elimina un fitxer.
cat fitxer	Mostra el contingut d'un fitxer.
utilitza la tecla ↑	Recuperar l'última comanda.
clear	Neteja la consola.

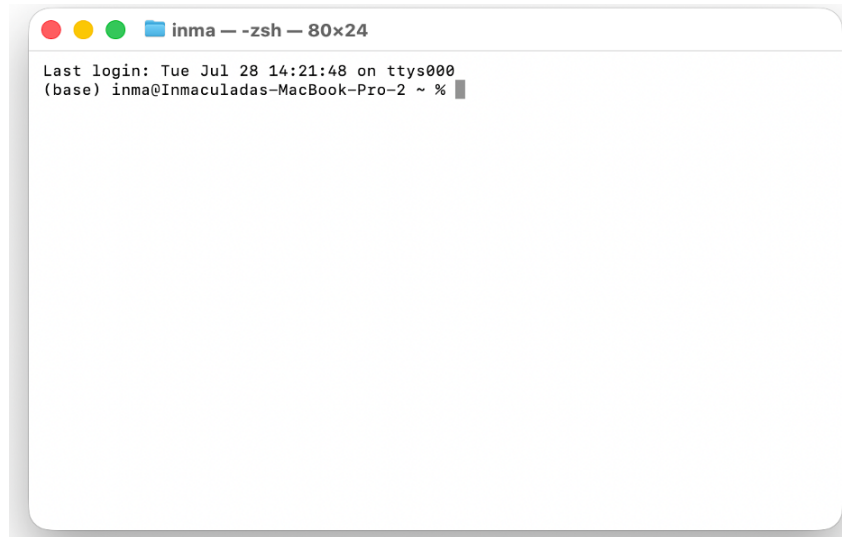


Figura 2: Consola

3.2 El directori actual

Quan s'obre la consola, sempre ens trobem situats en un directori determinat.

Per saber quin és el directori actual utilitzem la comanda:

```
1 || pwd
```

La sortida serà semblant a:

```
1 || _/home/alumne
```

o bé

```
1 || _/Users/alumne
```

Aquest és el directori HOME, on es crearan els fitxers si no indiquem cap altra ubicació.

3.3 Visualitzar el contingut d'un directori

La comanda

```
1 || ls
```

mostra els fitxers i directoris existents.

Si volem més informació podem utilitzar

```
1 || ls -l
```

i, si també volem veure els fitxers ocults,

```
1 || ls -la
```

La tecla ↑ per a **recuperar qualsevol comanda** que has escrit anteriorment. A partir d'ara l'utilitzaràs molt!!

3.4 Crear un directori

S'utilitza la comanda `mkdir`. Per exemple:

```
1 || mkdir _practical
```

A continuació podem comprovar que s'ha creat:

```
1 || ls
```

La tecla **Tab** serveix per **completar automàticament** comandes, fitxers i directoris. Per exemple, si tenim un directori anomenat `practica1` i escrivim `cd pra` podem prémer la tecla **Tab** i la consola completarà automàticament el nom. Prement dues vegades la tecla **Tab**, es mostren totes les opcions de completat disponibles.

3.5 Canviar de directori

Per entrar dins un directori utilitzem:

```
1 || cd practica1
```

Podem verificar que ens hi trobem executant:

```
1 || pwd
```

Per tornar al directori anterior:

```
1 || cd ..
```

Per tornar directament al directori personal:

```
1 || cd
```

3.6 Eliminar fitxers

Per eliminar un fitxer:

```
1 || rm hello.c
```

Atenció.

La comanda `rm` elimina definitivament el fitxer. No existeix una paperera de reciclatge des d'on recuperar-lo.

3.7 Visualitzar el contingut d'un fitxer

Sense necessitat d'obrir un editor podem mostrar el contingut d'un fitxer:

```
1 || cat hello.c
```

Aquesta comanda és especialment útil per comprovar ràpidament el contingut d'un fitxer petit. També podeu instal·lar l'utilitat `bat`, que amplia el comando `cat` amb ressaltat de sintaxis amb color, numeració de línies, etc.

3.8 Exercici consola

Realitzeu les operacions següents des de la consola:

1. Creeu un directori anomenat `intro_prog`.
2. Entreu dins aquest directori.
3. Creeu un directori anomenat `practica1`. És a dir, un subdirectori de `intro_prog`.
4. Entreu dins `practica1`.
5. Executeu `pwd`.
6. Torneu al directori anterior.
7. Executeu de nou `pwd`.
8. Comproveu el contingut del directori amb `ls -al`.

En la següent secció aprendrem a editar el nostre primer programa en C utilitzant l'editor `vim`.

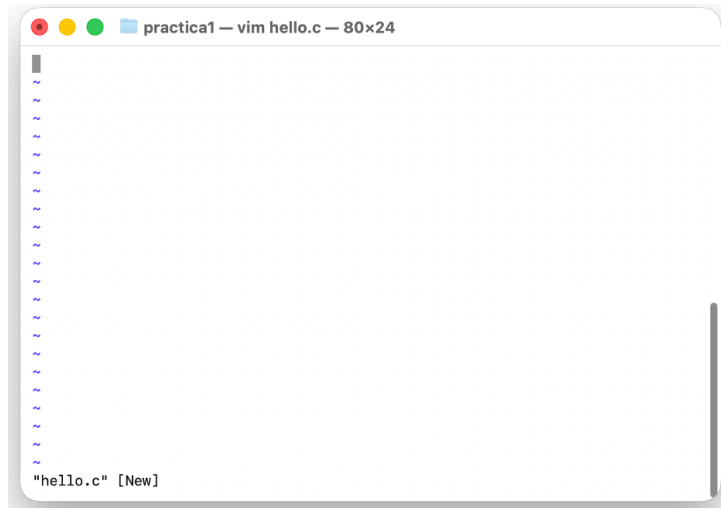


Figura 3: Editor vim

4 Edició de fitxers amb vim

Per escriure programes en C necessitem un editor de text. Durant les pràctiques utilitzarem **vim**, un editor molt lleuger que funciona directament des de la consola, però no és obligatori. Podeu utilitzar altre editor visual com ara el **gedit**.

Encara que al principi **vim** pot semblar diferent dels editors gràfics habituals, **vim** és una eina molt potent i està disponible en pràcticament qualsevol sistema Linux o Unix. En aquest tutorial només aprendrem les ordres bàsiques, necessàries per editar els nostres primers programes en C.

4.1 Crear un fitxer

Situeu-vos dins el directori de la pràctica:

```
1 || cd intro_prog/practica1
```

Per crear un fitxer anomenat **hello.c** executeu:

```
1 || vim hello.c
```

Com mostra la Figura 3, s'obrirà l'editor **vim**. Si el fitxer encara no existeix, es crearà automàticament quan el deseu (veure secció 4.3).

4.2 Els modes de vim

A diferència de la majoria d'editors, **vim** disposa de diferents modes de funcionament. En aquest curs només utilitzarem dos modes.

Mode normal

És el mode en què s'inicia **vim**. En aquest mode no podem escriure text; les tecles s'interpreten com a ordres. Sempre que tingueu dubtes sobre l'estat de l'editor, premeu la tecla **[Esc]**. Això us retornarà al mode normal.

Mode inserció

Per començar a escriure text cal entrar al mode inserció. Premeu: **[i]**. A la part inferior de la pantalla apareixerà: **-- INSERT --**. Ara ja podeu escriure el vostre programa.

4.3 Desar els canvis

Quan hàgiu acabat d'escriure:

1. Premeu **Esc**
2. Escriviu:

1 || :w

i premeu **Enter**

El fitxer quedarà desat.

4.4 Sortir de vim

Per sortir de l'editor:

1. Premeu **Esc**
2. Escriviu:

1 || :q

Si el fitxer ja estava desat, vim es tancarà.

4.5 Desar i sortir

L'opció més habitual és desar el fitxer i sortir alhora.

Per fer-ho:

1. Premeu **Esc**
2. Escriviu:

1 || :wq

i premeu **Enter**

4.6 Sortir sense desar

Si voleu abandonar l'editor descartant tots els canvis realitzats:

1. Premeu **Esc**
2. Escriviu:

1 || :q!

4.7 Ordres útils de vim

La Taula 1 resumeix algunes de les ordres més útils que utilitzarem al llarg del curs.

Ordre	Acció
i	Entrar al mode inserció.
Esc	Tornar al mode normal.
:w	Desar el fitxer.
:wq	Desar i sortir.
:q!	Sortir sense desar.
u	Desfer l'última acció (Undo).
o	Crear una nova línia sota l'actual i entrar al mode d'inserció.
Ctrl+r	Refer una acció desfeta (Redo).
Ctrl+c	Copia text seleccionat.
Ctrl+v	Paste text seleccionat
0	Anar a l'inici de la línia.
\$	Anar al final de la línia.
gg	Anar a l'inici del fitxer.
G	Anar al final del fitxer.
dd	Esborrar la línia actual.
yy	Copiar la línia actual.
p	Enganxar després del cursor.
/paraula	Cercar una paraula.
n	Anar a la següent coincidència, quan has fet abans /paraula.
N	Anar a la coincidència anterior, quan has fet abans /paraula.
:set number	Mostrar els números de línia.
:set nonumber	Ocultar els números de línia.

Taula 1: Ordres bàsiques de vim.

4.8 Exercici 1 vim

Realitzeu les operacions següents:

1. Creeu, al directori `intro_prog/practical1` el fitxer `hello.c` amb vim.
2. Escriviu qualsevol text, per exemple, "Contingut de hello.c".
3. Deseu el fitxer amb `:w`.
4. Sortiu amb `:q`.
5. Visualitzeu el contingut del fitxer mitjançant:

```
1 || cat hello.c
```

4.9 Exercici 2 vim

En aquest exercici practicarem més operacions bàsiques de vim: crear i editar un fitxer, moure'ns pel text, copiar, enganxar, eliminar línies i desar els canvis.

1. Aneu al directori `intro_prog/practical1` i creeu el fitxer `hello.c` amb vim:

```
1 || vim hello.c
2 ||
```

2. Entreu en el mode d'inserció prement la tecla i i escriviu el programa següent:

```

1 | #include <stdio.h>
2 |
3 | int main(void) {
4 |     printf("Hola, mon!\n");
5 |     return 0;
6 | }
7 |

```

3. Premeu **Esc** per tornar al mode normal.
4. Utilitzeu les tecles $\uparrow$, $\downarrow$, $\leftarrow$ i $\rightarrow$ per moure-us pel text.
5. Situeu-vos al final del programa i premeu **o**. Es crearà una nova línia sota la línia actual i entrareu en mode d'inserció. Escriviu:

```
/* Primer programa en C */
```

6. Premeu **Esc** per tornar al mode normal i deseu els canvis amb:

```

1 | :w
2 |

```

7. Situeu-vos sobre la línia que conté el comentari `/* Primer programa en C */` i utilitzeu **dd** per eliminar-la.
8. Desfeu l'operació anterior amb la tecla **u**.
9. Utilitzeu **yy** per copiar la línia del comentari i, a continuació, utilitzeu **p** per enganxar-la a la línia següent.
10. Elimineu una de les dues línies del comentari amb **dd**.
11. Deseu els canvis amb **:w**.
12. Sortiu de **vim** amb:

```

1 | :q
2 |

```

13. Visualitzeu el contingut del fitxer amb:

```

1 | cat hello.c
2 |

```

14. Torneu a obrir el fitxer amb **vim** i practiqueu la navegació pel text. Proveu també de tornar al mode normal amb **Esc** després d'entrar en el mode d'inserció.
15. Finalment, deseu els canvis i sortiu de **vim** amb:

```

1 | :wq
2 |

```

5 Instal·lació del compilador de C

Per poder escriure i executar programes en C és necessari disposar d'un **compilador**. Al llarg de l'assignatura utilitzarem **gcc** (GNU Compiler Collection), encara que els exemples d'aquest tutorial també funcionen amb **clang**, el compilador que s'utilitza habitualment a macOS.

Abans de començar a programar és recomanable comprovar si el compilador ja està instal·lat al vostre ordinador.

5.1 Comprovar la instal·lació

Obriu una consola i executeu la comanda:

```
1 || gcc --version
```

Si el compilador està instal·lat, obtindreu una sortida semblant a:

```
1 || gcc (Ubuntu) 13.3.0
2 || Copyright (C) 2023 Free Software Foundation, Inc.
3 || ...
```

Si la comanda anterior mostra un missatge indicant que el compilador no existeix, haureu d'instal·lar-lo. A continuació, expliquem com fer-ho segons el teu sistema operatiu.

5.2 Linux

En la majoria de distribucions Linux el compilador es pot instal·lar des del gestor de paquets.

En distribucions basades en Ubuntu o Debian:

```
1 || sudo apt update
2 || sudo apt install build-essential
```

El paquet `build-essential` instal·la el compilador `gcc` i altres eines bàsiques que utilitzarem durant el curs.

Una vegada finalitzada la instal·lació, torneu a executar:

```
1 || gcc --version
```

per comprovar que el compilador està disponible.

5.3 macOS

A macOS el compilador forma part de les *Command Line Tools* d'Apple. Si encara no les teniu instal·lades, executeu:

```
1 || xcode-select --install
```

Quan finalitzi la instal·lació, podeu comprovar-ho amb:

```
1 || clang --version
```

o bé

```
1 || gcc --version
```

Observeu que, en macOS, la comanda `gcc` acostuma a executar internament el compilador `clang`.

5.4 Windows

Als ordinadors amb Windows recomanem instal·lar el compilador mitjançant **MSYS2** <https://www.msys2.org/>. MSYS2 proporciona un entorn molt similar al que utilitzarem als laboratoris de la universitat i inclou el compilador GCC.

Una vegada instal·lat, obriu la consola *MSYS2 UCRT64* i executeu:

```
1 || gcc --version
```

Si el compilador està correctament instal·lat, apareixerà la seva versió.

Com que ja disposem d'un compilador instal·lat, sabem moure'ns per la consola i podem editar fitxers amb `vim`. Ha arribat el moment d'escriure el nostre primer programa en llenguatge C.

6 El primer programa en C

Tradicionalment, el primer programa que s'escriu en qualsevol llenguatge és el programa *Hello World*, que simplement mostra un missatge per pantalla. Anem a veure amb un exemple les etapes introduïdes al començament del tutorial.

6.1 Etapa d'EDICIÓ: crear i editar el fitxer font

Situeu-vos dins del directori de la primera pràctica:

```
1 || cd ~ / intro_prog / practica1
```

Nota: El símbol `~` representa el directori personal de l'usuari. Per exemple, `~/intro_prog/practica1` és equivalent a `/home/usuari/intro_prog/practica1`.

Creeu un nou fitxer anomenat `hello.c`. Si heu fet l'exercici 2, obriu-lo:

```
1 || vim hello.c
```

Escriviu el següent programa.

```
1 || #include <stdio.h>
2 ||
3 || int main(void)
4 || {
5 ||     printf("Hola, món!\n");
6 ||     return 0;
7 || }
```

Deseu el fitxer amb:

```
1 || :wq
```

Vegem què significa cadascuna de les línies.

La primera línia incorpora la biblioteca estàndard d'entrada i sortida del llenguatge C.

```
1 || #include <stdio.h>
```

Aquesta biblioteca conté, entre d'altres, la funció `printf()`, que utilitzarem per escriure text a la pantalla.

Tot programa en C comença la seva execució a la funció `main`.

```
3 || int main(void)
```

La paraula `int` indica que aquesta funció retorna un nombre enter al sistema operatiu.

L'expressió `void` indica que la funció no rep cap paràmetre.

Les claus

Les instruccions que formen part de la funció s'escriuen entre claus.

```
4 || {
5 ||
6 ||
7 || }
```

Totes les instruccions situades dins d'aquestes claus s'executaran quan el programa comenci.

La funció `printf()` permet escriure text per pantalla.

```
5 || printf("Hola, món!\n");
```

Observeu que el text apareix entre cometes dobles, totes les instruccions acaben amb punt i coma (`;`), i `\n` indica un salt de línia.

L'última instrucció és

```
6 || return 0;
```

Aquesta instrucció finalitza el programa i indica al sistema operatiu que l'execució ha acabat correctament.

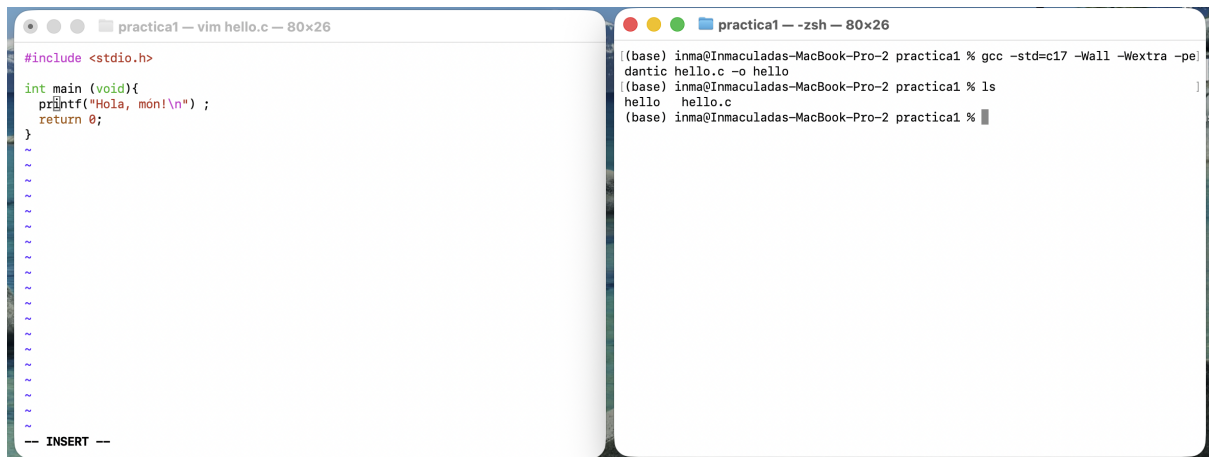


Figura 4: Dues consoles: a l'esquerra per editar, a la dreta per compilar i executar.

6.2 Etapa de COMPILACIÓ

Per convertir el codi font en un programa executable utilitzarem el compilador `gcc`. El llenguatge C ha evolucionat al llarg dels anys i existeixen diferents versions de l'estàndard, com ara C89, C99, C11 i C17. En aquest tutorial utilitzarem C17, publicat l'any 2018, perquè és una revisió de C11 que incorpora correccions i és àmpliament suportada pels compiladors actuals. Per aquest motiu compilarem sempre els programes amb l'opció `-std=c17`. Això garanteix que tots els alumnes treballin amb la mateixa versió del llenguatge, independentment del sistema operatiu o del compilador utilitzat.

Al mateix directori on teniu el fitxer font `hello.c`, executeu la següent comanda:

```
1 || gcc -std=c17 -Wall -Wextra -pedantic hello.c -o hello
```

Si no apareix cap missatge, significa que el programa s'ha compilat correctament.

Recomanem obrir dues consoles (veure Figura 4), una al costat de l'altra. Una per editar el codi font i una altra per compilar-lo i executar-lo. D'aquesta manera, podreu veure fàcilment la sortida de la compilació i de l'execució alhora que teniu el codi font disponible per editar-lo en cas que hi hagi errors.

Podeu comprovar que s'ha creat un nou fitxer executant:

```
1 || ls
```

La sortida serà semblant a:

```
1 || hello
2 || hello.c
```

El fitxer `hello` és el programa executable.

Vegem què significa cadascuna de les opcions utilitzades.

Opció	Descripció
<code>-std=c17</code>	Compila el programa utilitzant l'estàndard C17.
<code>-Wall</code>	Activa les advertències més habituals del compilador.
<code>-Wextra</code>	Activa advertències addicionals que ajuden a detectar possibles errors.
<code>-pedantic</code>	Comprova que el programa compleix estrictament l'estàndard del llenguatge C.
<code>-o hello</code>	Indica que el fitxer executable s'anomenarà <code>hello</code> .

Durant tota l'assignatura utilitzarem aquestes opcions en totes les compilacions.

6.3 Etapa d'EXECUCIÓ

Una vegada compilat el programa, podem executar-lo.

A Linux i macOS:

```
1 || ./hello
```

Apareixerà:

```
1 || Hola, mon!
```

En Windows (MSYS2) també podeu executar amb:

```
1 || ./hello.exe
```

6.4 Errors

Si es produeixen errors, no es genera el fitxer objecte (.o) i hem, per tant, de fixar-lo abans de continuar. Suposem que oblidem escriure el punt i coma.

```
1 || printf("Hola, mon!\n")
```

En intentar compilar apareixerà un missatge semblant a:

```
1 || error: expected ';' before 'return'
```

Quan el compilador mostra un error llegiu atentament el missatge, localitzeu la línia indicada, corregiu l'error i torneu a compilar.

6.5 Advertències (warnings)

No totes les incidències són errors. Per exemple, si es crea una variable però després no s'utilitza, alguns compiladors (com Apple Clang) mostren una advertència (*warnings*). Per exemple:

```
1 || int main(void)
2 || {
3 ||     int x = 10;
4 ||     printf("Hola, mon!\n");
5 ||     return 0;
6 || }
```

En aquest cas, el compilador mostra un *warnings* com:

```
1 || warning: unused variable 'x' [-Wunused-variable]
```

El programa es compila correctament, però el compilador avisa que la variable “x” s’ha declarat i no s’utilitza (més endavant veurem com es creen i s'utilitzen les variables).

Amb els *warnings* es pot generar el fitxer objecte, però amb els errors no, s’han de fixar abans de poder continuar.

6.6 Exercicis

1. Modifiqueu el programa perquè mostri el vostre nom. Compila i executa.
2. Feu que mostri el vostre nom en una línia, i els vostres cognoms en la següent línia utilitzant `\n`. Compila i executa.
3. Introduïu intencionadament un error de sintaxi i observeu el missatge del compilador.
4. Corregiu l'error i torneu a compilar.

7 Funcions

Fins ara l'únic programa que hem fet només contenia una única funció, la funció principal, `main()`. Tot programa en C ha de tenir una única funció `main()`, on comença (i termina) l'execució del programa.

Tanmateix, a mesura que els programes creixen, escriure tot el codi dins de `main()` fa que siguin difícils de llegir i mantenir. Per aquest motiu, C, com la majoria de llenguatges de programació, permet dividir un programa en **funcions**.

Concretament, una funció és un conjunt d'instruccions que realitza una tasca concreta. Les funcions presenten nombrosos avantatges, eviten repetir codi, fan que els programes siguin més fàcils de llegir, permeten dividir un problema gran en problemes més petits, faciliten la reutilització del codi, i simplifiquen el manteniment dels programes.

I molt important! Una mateixa funció es pot executar tantes vegades com sigui necessari! Anem a escriure la nostra primera funció en C.

7.1 La nostra primera funció en C

Crearem una funció anomenada `saludar()`. Editeu el programa anterior, `hello.c`, de la següent forma.

```
1 | #include <stdio.h>
2 |
3 | void saludar(void)
4 | {
5 |     printf("Hola, món!\n");
6 | }
7 |
8 | int main(void)
9 | {
10 |     saludar();
11 |
12 |     return 0;
13 | }
```

Compileu el programa com sempre:

```
1 | gcc -std=c17 -Wall -Wextra -pedantic hello.c -o hello
```

Executeu-lo:

```
1 | ./hello
```

Obtindreu exactament la mateixa sortida que abans.

Expliquem una mica la funció `saludar()`. Aquesta és la **definició** de la funció:

```
3 | void saludar(void)
4 | {
5 |     printf("Hola, món!\n");
6 | }
```

Està formada per:

- el nom de la funció. És el nom que utilitzarem sempre que vulguem executar-la. Com qualsevol identificador en C, ha de començar per una lletra o el caràcter `_` i no pot contenir espais.
- el tipus de retorn. La paraula `void` a l'esquerra del nom indica que aquesta funció no retorna cap valor. Més endavant veurem funcions que retornen enters, nombres reals o altres tipus de dades.
- la llista de paràmetres. Entre els parèntesis apareix `void`. Això significa que la funció no necessita rebre cap dada per poder executar-se. Més endavant aprendrem a definir funcions que reben informació mitjançant paràmetres.
- el cos de la funció. És el conjunt d'instruccions que hi ha entre les claus `{}`.

7.2 Cridar una funció

Des de `main()` podem executar una funció escrivint-ne el nom.

```
10 || saludar();
```

Aquest procés rep el nom de **crida a una funció**. Quan el programa arriba a aquesta instrucció es comença a executar la funció. Quan finalitza, el programa continua a la instrucció següent després de la crida a la funció.

Una mateixa funció es pot executar més d'una vegada. Per exemple:

```
1 || int main(void)  
2 || {  
3 ||     saludar();  
4 ||     saludar();  
5 ||     saludar();  
6 ||   
7 ||     return 0;  
8 || }
```

La sortida serà:

```
1 || Hola, món!  
2 || Hola, món!  
3 || Hola, món!
```

Observeu que no ha estat necessari copiar el codi tres vegades.

7.3 Exercicis

1. Creeu una funció anomenada `acomiarar()` que mostri el missatge:

```
1 || Fins aviat!
```

i crideu-la des de `main()`.

2. Modifiqueu el programa perquè `main()` cridi dues vegades a `saludar()` i una vegada a `acomiarar()`.
3. Creeu una tercera funció que mostri el vostre nom.

8 Organització del codi en mòduls

Fins ara tots els programes s'han escrit en un únic fitxer. Aquesta és una bona estratègia quan els programes són petits, però a mesura que augmenta la seva complexitat resulta més difícil localitzar les funcions i mantenir el codi. Una manera d'organitzar millor els programes consisteix a dividir-los en **mòduls**.

Un mòdul agrupa funcions que realitzen una mateixa tasca. Per exemple, en un programa gran podríem tenir un mòdul per gestionar la pantalla, un altre per realitzar càlculs, etc. Separar el programa en mòduls presenta diversos avantatges.

- El codi és més fàcil de llegir.
- Cada fitxer té una única responsabilitat.
- Les funcions es poden reutilitzar en altres programes.
- Diversos programadors poden treballar sobre fitxers diferents.
- Els projectes grans són més fàcils de mantenir.

8.1 Els fitxers d'un mòdul

Habitualment un mòdul està format per dos fitxers.

- Un fitxer `.h` (capçalera).
- Un fitxer `.c` (implementació).

Per exemple:

```
salutacio.h  Declaració d'una o més funcions.
salutacio.c  Implementació d'una o més funcions.
```

El programa principal continuarà estant al fitxer `main.c`. Per tant, l'estructura del projecte serà:

```
practical/
|
+-- main.c
+-- salutacio.c
+-- salutacio.h
```

La Taula 2 mostra la funció de cada fitxer.

Fitxer	Funció
main.c	Conté la funció principal del programa. Usa la funció <code>saludar()</code> .
salutacio.h	Declara les funcions disponibles del mòdul. En aquest exemple <code>saludar()</code> .
salutacio.c	Implementa les funcions declarades a la capçalera. En aquest cas, <code>saludar()</code> .

Taula 2: Funció de cada fitxer del mòdul

8.2 La capçalera

Creem primer el fitxer `salutacio.h` amb el contingut següent.

```
1 | #ifndef SALUTACIO_H
2 | #define SALUTACIO_H
3 |
4 | void saludar(void);
5 |
6 | #endif
```

Aquest fitxer conté únicament la declaració de la funció. Observeu que no apareix el cos de la funció.

Nota: La seqüència `#ifndef`, `#define` i `#endif` s'utilitza com a guarda d'inclusió (*include guard*) per evitar que un fitxer de capçalera es processi més d'una vegada.

Es pot llegir de la manera següent: `#ifndef SALUTACIO_H` significa “si `SALUTACIO_H` no està definit”, `#define SALUTACIO_H` el defineix, i el contingut del fitxer només es processa en aquest cas. Si el fitxer es torna a incloure més endavant, `SALUTACIO_H` ja estarà definit i el preprocessador n'ignorarà el contingut, evitant errors de redefinició.

Com a regla pràctica, el nom de la guarda d'inclusió es basa en el nom del fitxer de capçalera. Per exemple, per a `salutacio.h`, podem utilitzar `SALUTACIO_H`. És important que el nom sigui prou específic i no coincideixi amb el d'una altra guarda del projecte.

8.3 La implementació

Ara creem el fitxer `salutacio.c`.

```
1 | #include <stdio.h>
2 | #include "salutacio.h"
3 |
4 | void saludar(void)
5 | {
6 |     printf("Hola, món!\n");
7 | }
```

Aquí sí que apareix el codi de la funció. Aquest fitxer rep el nom d'**implementació**.

Nota: En una directiva `#include`, les cometes angulars `< >` s'utilitzen habitualment per incloure fitxers de capçalera estàndard o instal·lats pel sistema, com ara `<stdio.h>`. En canvi, les cometes dobles `" "` s'utilitzen habitualment per incloure fitxers de capçalera propis del projecte, com ara `"salutacio.h"`. Per tant:

- `#include <stdio.h>` → busca el fitxer en els directoris del sistema.
- `#include "salutacio.h"` → busca primer el fitxer en els directoris del projecte.

8.4 El programa principal

Finalment escrivim el fitxer `main.c`.

```
1 | #include "salutacio.h"
2 |
3 | int main(void)
4 | {
5 |     saludar();
6 |
7 |     return 0;
8 | }
```

Observeu que `main.c` no coneix la implementació de la funció. Només necessita saber que existeix. Aquesta informació la proporciona la capçalera (fitxer `.h`).

8.5 Compilar els fitxers del mòdul

Ara el programa està repartit en dos fitxers font. Per tant, tots dos s'han de compilar.

```
1 | gcc -std=c17 -Wall -Wextra -pedantic \
2 |    main.c salutacio.c -o programa
```

Si la compilació finalitza correctament apareixerà un nou executable **programa** que podrem executar amb

```
1 | ./programa
```

La sortida serà:

```
1 | Hola, món!
```

9 Compilació separada

En la secció anterior hem vist com compilar un programa format per diversos fitxers. Per exemple:

```
1 | gcc -std=c17 -Wall -Wextra -pedantic \
2 |    main.c salutacio.c -o programa
```

Aquesta ordre és suficient per a programes petits. No obstant això, en projectes grans pot resultar poc eficient. Imaginem un projecte amb cent fitxers font. Si només modifiquem un únic fitxer, no és necessari tornar a compilar-los tots. Per aquest motiu, els compiladors permeten realitzar una **compilació separada** (veure Figura 5).

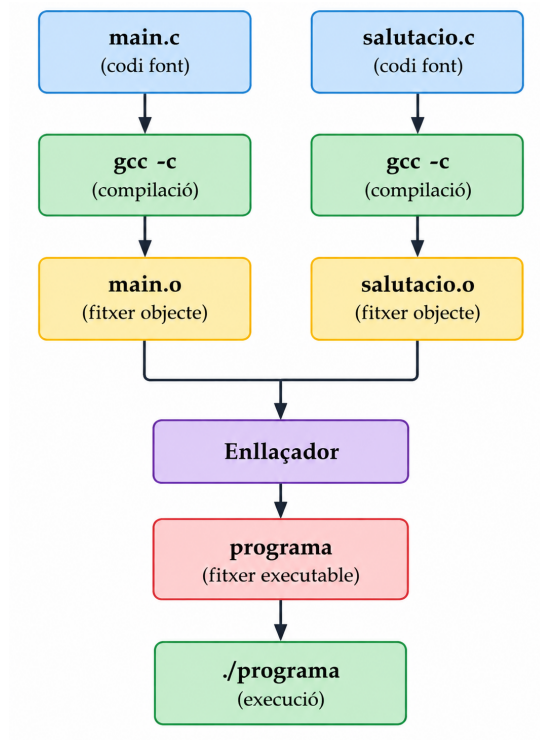


Figura 5: Compilació 2 mòduls

9.1 Els fitxers objecte

La compilació separada consisteix a compilar cada fitxer font de manera independent.

Per fer-ho utilitzem l'opció:

```
1 || -c
```

Per exemple:

```
1 || gcc -std=c17 -Wall -Wextra -pedantic \
2 || -c main.c
```

Aquesta ordre genera el fitxer

```
1 || main.o
```

Sense crear encara el programa executable.

De manera similar:

```
1 || gcc -std=c17 -Wall -Wextra -pedantic \
2 || -c salutacio.c
```

genera

```
1 || salutacio.o
```

Si executem

```
1 || ls
```

obtindrem una sortida semblant a:

```
1 || main.c
2 || main.o
3 ||
4 || salutacio.c
5 || salutacio.h
6 || salutacio.o
```

Observeu que encara no existeix el programa executable.

9.2 L'enllaçat

Una vegada disposem de tots els fitxers objecte, podem construir l'executable.

Per fer-ho executem:

```
1 || gcc main.o salutacio.o -o programa
```

Ara sí que apareixerà un nou fitxer:

```
1 || programa
```

que podrem executar com sempre:

```
1 || ./programa
```

La Taula 3 resumeix les diferents ordres que hem vist quan fem compilació conjunta i compilació separada.

Ordre	Acció
Compilació conjunta	
gcc main.c salutacio.c -o programa	Compila i enllaça en una sola ordre.
Compilació separada	
gcc -c main.c	Genera el fitxer objecte <code>main.o</code> .
gcc -c salutacio.c	Genera el fitxer objecte <code>salutacio.o</code> .
gcc main.o salutacio.o -o programa	Enllaça els fitxers objecte i crea l'executable.

Taula 3: Ordres de compilació.

En la següent secció automatitzarem tot aquest procés mitjançant `make` i els fitxers `Makefile`.

10 Automatització de la compilació amb make

En la secció anterior hem vist que un programa format per diversos fitxers es pot compilar de manera separada. Aquest procediment funciona correctament, però resulta repetitiu. Cada vegada que es modifica un fitxer cal tornar a escriure aquestes ordres. Per automatitzar aquesta tasca s'utilitza l'eina `make`.

10.1 Què és un Makefile?

Un `Makefile` és un fitxer de text que descriu com s'ha de construir un programa. Conté les ordres necessàries per compilar els fitxers font i generar l'executable.

Quan executem l'ordre `make`, el programa busca un fitxer anomenat `Makefile` i executa les regles que hi troba. Una regla d'un `Makefile` indica quin objectiu es vol construir, de quins fitxers depèn i quina ordre s'ha d'executar per construir-lo. Per exemple, la regla `programa: main.o salutacio.o` indica que l'objectiu `programa` depèn de `main.o` i `salutacio.o`. Per tant, anirà a les regles (`main.o`: i `salutacion.o`;) que permeten generar-los.

10.2 El nostre primer Makefile

Creeu un fitxer anomenat `Makefile` amb el contingut següent.

```
1 | CC=gcc
2 | CFLAGS=-std=c17-Wall-Wextra-pedantic
3 |
4 | programa: main.o salutacio.o
5 |          $(CC) main.o salutacio.o -o programa
6 |
7 | main.o: main.c salutacio.h
```

```

8 |  _ _ _ _ _ _ _ _ $(CC) _ $(CFLAGS) _ -c _ main.c
9 |
10 | salutacio.o: _ salutacio.c _ salutacio.h
11 |  _ _ _ _ _ _ _ _ $(CC) _ $(CFLAGS) _ -c _ salutacio.c

```

Atenció: les línies que contenen les ordres de compilació (a l'exemple: 5, 8, 11) han de començar amb un caràcter de tabulació (**Tab**) i no amb espais.

Nota: En un Makefile, la primera regla defineix l'objectiu predeterminat que s'executa quan s'invoca l'ordre **make** sense especificar cap objectiu. Per aquest motiu, **programa** apareix en primer lloc. Tot i això, l'ordre de les regles no determina l'ordre de compilació. **make** analitza les dependències de cada objectiu i construeix primer els fitxers necessaris. En aquest cas, abans de generar l'executable **programa**, es compilen **main.o** i **salutacio.o**, ja que són dependències directes d'aquest objectiu.

10.3 Compilar el projecte

Una vegada desat el fitxer només cal executar `make` i obtindrem una sortida semblant a:

```
1 gcc_std=c17_Wall_Wextra_pedantic_c_main.c
2 gcc_std=c17_Wall_Wextra_pedantic_c_salutacio.c
3 gcc_main.o_salutacio.o_o_programa
```

Finalment apareixerà l'executable **programa**. Recordeu que un dels principals avantatges de **make** és que només recompila els fitxers necessaris. Per exemple, si només modifiquem **salutacio.c** i tornem a executar **make** només es mostrarà el que veieu a continuació (observeu que **main.c** no s'ha recompilat):

```
1 gcc_std=c17-Wall-Wextra-pedantic-c_salutacio.c
2 gcc_main.o_salutacio.o-o_programa
```

10.4 Netejar el projecte

Després de diverses compilacions s'acumulen fitxers objecte. És habitual afegir una regla per eliminar-los. Afegiu al final del **Makefile**:

```
1 | clean:
2 | rm -f *.o programa
```

Ara podrem executar `make clean` per eliminar els fitxers objecte i l'executable.

11 Depuració bàsica

Un programa pot compilar correctament i, tot i així, no produir el resultat esperat. En aquest cas cal localitzar l'error. Aquest procés rep el nom de **depuració**.

11.1 Utilitzar printf()

Una manera senzilla de saber què està fent un programa és mostrar informació per pantalla.

Per exemple:

```
1 printf("Comença el programa\n");
2 printf("La variable edat val: %d", edat);
```

o bé

```
1 || printf("He entrat a la funció saludar()\n");
```

Aquests missatges permeten seguir el recorregut del programa. Quan el problema s'ha resolt, aquests missatges es poden eliminar.

Cal dir que les IDEs (Integrated Development Environment) tenen eines visuals molt potents per a fer debugging.

12 Guia d'estil

Un programa no només ha de funcionar correctament, sinó que també ha de ser fàcil de llegir i de mantenir. De fet, la qualitat d'un programa no depèn únicament del fet que funcioni. Un codi clar, ben estructurat i coherent és més fàcil d'entendre, mantenir i modificar, tant per al seu autor com per a qualsevol altre programador.

Podem comparar un programa amb un llibre. Si l'autor escriu amb claredat, els lectors entendran millor el contingut. De la mateixa manera, un codi ben escrit facilita que altres programadors puguin comprendre'l, modificar-lo i detectar possibles errors.

Les guies d'estil de programació són un conjunt de recomanacions que ajuden a escriure programes de manera coherent i llegible. Encara que no siguin obligatòries, és una bona pràctica seguir-les sempre. La guia d'estil d'aquesta assignatura recomana:

- Escriure comentaris quan siguin necessaris.
- Utilitzar noms descriptius.
- Mantenir un format uniforme.
- Fer servir una notació consistent.

12.1 Comentaris

Els comentaris serveixen per documentar el codi. No són processats pel compilador, però ajuden tant a l'autor del programa com a qualsevol altra persona que l'hagi de llegir en el futur.

És recomanable escriure comentaris:

- al començament de cada fitxer;
- abans de funcions importants;
- abans de blocs de codi difícils d'entendre.

No és convenient comentar instruccions que ja són evidents. Per exemple, aquest comentari no aporta cap informació:

```
1 || int edat; /* Guarda l'edat */
```

En canvi, aquest comentari sí que explica la finalitat de la funció.

```
1 || /*  
2 ||  * Mostra un missatge de benvinguda.  
3 ||  */  
4 || void saludar(void)  
5 || {  
6 ||     printf("Hola!\n");  
7 || }
```

12.2 Noms amb sentit

Els noms de les variables i de les funcions han de descriure clarament el seu propòsit. Per exemple,

```
1 || int d;
```

és molt menys clar que

```
1 || int nombre_dies;
```

Els noms han de permetre entendre el programa sense necessitat de llegir-ne tots els detalls.

També és convenient evitar noms massa semblants entre ells.

Per exemple,

```

1 | int_l;
2 | int_I;

```

poden resultar fàcils de confondre.
És recomanable que les variables representin substantius.

```

1 | int_edat;
2 | float_temperatura;
3 | int_nombre_alumnes;

```

En canvi, les funcions representen accions i és preferible utilitzar verbs.

```

1 | calcular_mitjana();
2 | llegir_fitxer();
3 | mostrar_menu();

```

12.3 Format del codi

Un format coherent facilita molt la lectura dels programes. Observeu la diferència entre els dos exemples següents.

```

1 | int main(void){
2 | int a=10,b=20;
3 | printf("Hola\n");
4 | printf("%d\n",a+b);
5 | return 0;}

```

```

1 | int main(void)
2 | {
3 |     int a = 10;
4 |     int b = 20;
5 |
6 |     printf("Hola\n");
7 |     printf("%d\n", a + b);
8 |
9 |     return 0;
10 | }

```

El segon programa és molt més fàcil de llegir. Per aquest motiu procurarem seguir sempre aquestes recomanacions:

- utilitzar una indentació coherent;
- escriure una instrucció per línia;
- deixar espais al voltant dels operadors;
- separar amb línies en blanc els blocs de codi que realitzen tasques diferents.

També és convenient que les línies no siguin excessivament llargues. Sempre que sigui possible, intentarem que no superin aproximadament els 80 caràcters.

12.4 Notació consistent

El llenguatge C no defineix una convenció oficial per anomenar variables, funcions o fitxers.

Per aquest motiu, cada projecte utilitza la seva pròpia guia d'estil. En aquesta assignatura seguirem una convenció única i consistent.

- Els noms de les variables i de les funcions començaran amb lletra minúscula.
- Si un nom està format per diverses paraules, s'utilitzarà la notació *snake_case*, és a dir, les paraules se separaran mitjançant el caràcter `_`.
- Les constants s'escriuran amb lletres majúscules i paraules separades mitjançant el caràcter `_`.
- Els noms dels fitxers `.c` i `.h` també utilitzaran la notació *snake_case*.

Per exemple,

```
1 | const int MAX_ALUMNES = 100;
2 |
3 | int nombre_alumnes;
4 |
5 | float nota_mitjana;
6 |
7 | void calcular_mitjana(void);
8 |
9 | void mostrar_menu(void);
```

Els fitxers del projecte podrien tenir noms com:

```
1 | main.c
2 | gestio_alumnes.c
3 | gestio_alumnes.h
```

L'objectiu no és trobar la millor convenció possible, sinó utilitzar sempre la mateixa dins d'un mateix projecte. Un programa escrit de manera coherent és més fàcil de llegir, mantenir i ampliar.